Concrete Smart arithmetic on a 160-bit fixture

The fixed finite-ring arithmetic program returns the certified logarithm in all four prescribed coefficient lifts. The Lean proof checks eight 211-node traces, all 1,680 polynomial transitions, public Hensel lifting, terminal inverse products, coefficient extraction and recovery. The final theorem binds the seven public integers to actual Mathlib curve points, proves the returned scalar satisfies dG=Q, and retains exact prime order p and 160-bit bounds. Whole gates pass 28 predata and 51 concrete declarations, with zero failing proofs and zero trusted-native exceptions. [pcc:unverified]

$$p = 730750818665451459112596905638433048232067471723,$$
$$d = 210393287966660756596132643172172640405085536179.$$

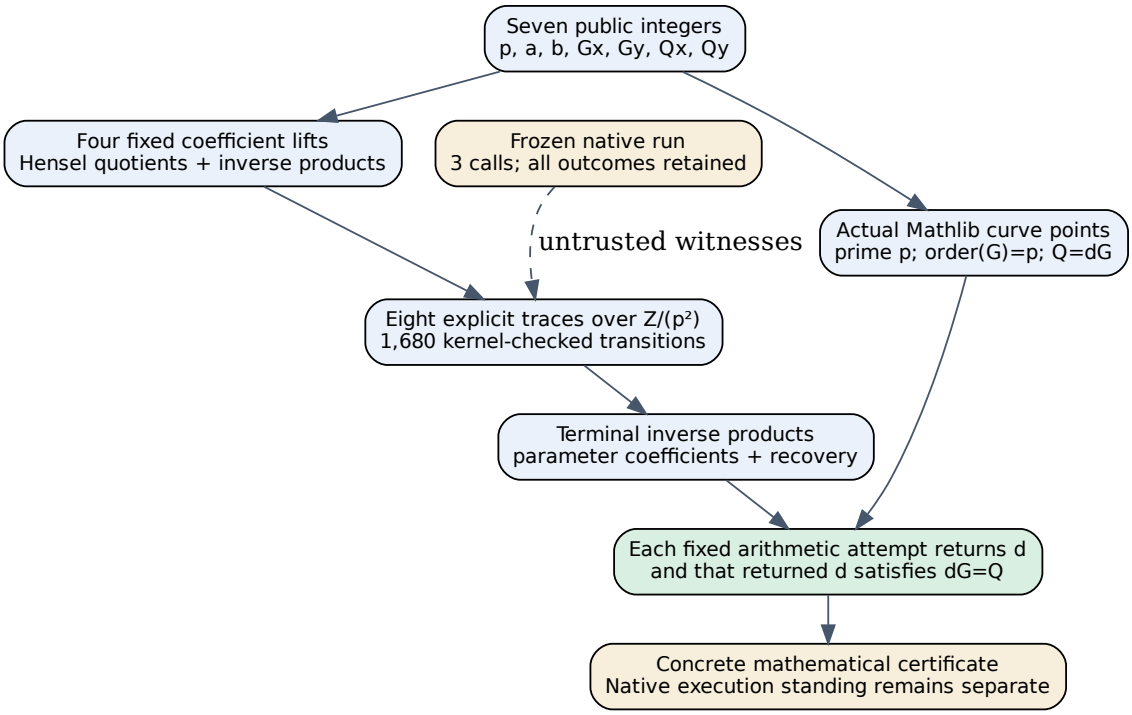


Figure 1: Explicit proof map; dashed edge supplies untrusted numerical witnesses. [pcc:unverified]

`SmartConcreteAudit.concrete_attack_certificate` is concrete arithmetic correctness with an actual point relation. It does not prove the generic p-adic transfer theorem, whole-curve cardinality, native compiler refinement, attack complexity or the SafeCurves constant. NIST explicitly excludes anomalous curves; its screened recommendation is not refuted. [pcc:unverified]

Frozen native run and exact evidence

Protocol `smart-known-answer-20261110/v1` was frozen at 08:56:55Z, hash `00e7a4a7...684c56`; source `70aa9f2f` and protocol commit 37302ba3 precede the three registered calls. Host 762f5fc120c9; existing shared local CPU, one native thread, no claimed timing isolation or accepted wall metric. Small preflights are explicitly in-sample; the large target was unseen at freeze. All four fixed lifts are retained, without retries or exclusions. Flow: 3 planned,3 attempted,3 completed,0 excluded/interrupted/resumed. [pcc:unverified]

Arm	Exit	Actual outcome	Logical operations
Small F5	0	Four nondegenerate results 2	24 total
Off-curve F5	2	Expected input rejection	No multiplication trace
160-bit fixture	0	Four nondegenerate results d	1,680 total

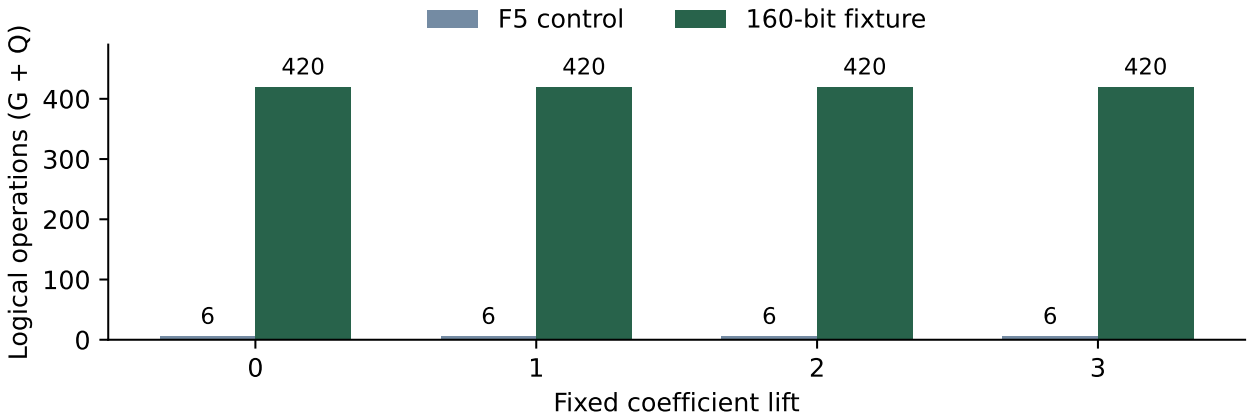


Figure 2: Recorded operator counts per fixed lift, across G and Q. Different inputs; no timing or speedup inference. [pcc:unverified]

The binary reads only p,a,b,Gx,Gy,Qx,Qy. Lifts are exactly A=a+ip, B=b+(2i+1)p for i=0,1,2,3. It computes p times the lifted points, evaluates -XZ/Y modulo p^2 and divides the coefficients of p. It receives no known scalar. Producer known-answer and shape checks pass; the export passes canonical file/schema/protocol validation with zero findings. Native packet `PCC-20261110-162b83` is locally `not_covered`, signed `unverified`, lacking independent signed replay/recomputation. Lean's mathematical proof does not certify that native execution. [pcc:unverified]

Proof sources, controls and reproducibility

Source	Obligation
Frozen `lean/SmartArithmetic`	Polynomial program, binary schedule and trace soundness
Frozen `lean/SmallControl`	Actual F5 point order 5, 2G=Q, off-curve equation disproof
`certificate-lean/SmartSpecification`	Public arithmetic attempt and witness composition
`SmartLargeLift{0..3}{G,Q}`	Every concrete transition, lift, coefficient and endpoint equation
`SmartLargeRecovery`	Four actual returned values
`SmartCertificate`	Fixed lift policy, public coordinates, actual returned logarithm

The new source formatter copies decimal strings; it performs no curve arithmetic. Kernel arithmetic checks every supplied numerical equality. Lean 4.35.0 / Mathlib 56a00585 are pinned, using only propext, Classical.choice and Quot.sound. No holes, custom axioms, native_decide exception, linter waiver or added elaboration setting. Full gate and source/packet bindings: Whole gates pass 28 predata and 51 concrete declarations, with zero failing proofs and zero trusted-native exceptions. Source commit `7fc27f7f` precedes theorem packet `PCC-20261110-d8466d`. Full local verification passes at 09:44:26Z; signed standing remains unverified.. [pcc:unverified]

The original program, protocol, analyzer, all raw runs and export remain unchanged. An initial attempt to add roots to the old Lake configuration was stopped when its extra export pin was found; exact pinned bytes were restored and canonical file verification passed. New proofs live in a separate dependent project. Failed proof sources/logs and the interrupted setup are preserved losslessly. Lean predicate/cast elaboration failures were repaired without changing any arithmetic statement or input. [pcc:unverified]

The parent certificate `PCC-20261110-90623b` proves the actual prime-order points and relation. Its source-j disproof reproduces the author's known erratum, not a new correction. Sources read: Smart HPL-97-128; EFD short-Weierstrass Jacobian add-1998-cmo/dbl-1998-cmo; Leprévost et al., IPL 93(5) 225–230 (2005), DOI 10.1016/j.ipl.2004.11.008 and author erratum; NIST SP 800-186 section 3.1.2/Appendix C.1. KB/Conductor unavailable; primary/repository/Mathlib fallback, no novelty claim. [pcc:unverified]

See `../../README.md`, `../../protocol.json`, `../../export.json`, `../../smart-sources.json`, `../../smart-verification.json` and preserved `../../evidence/`. No paid compute, security-cost claim or leaderboard promotion. The fourteen-item audit remains partial in the canonical census README; full generic/qualified results and actual separate-host validation remain separate obligations. [pcc:unverified]